

RAG

Retreival Augemented Genration



AHMAD JABIR DADA

The Problem

Current LLMs can answer general questions with ease, but when it comes to company specific questions or data unavailable to it they may:

- Hallucinate unsupported answers
- Use outdated data which is part of its original training dataset
- Fail to cite the correct internal document
- Answer from general knowledge instead of the document provided

The Solution

To solve the given problem, I built a local chatbot that answers from a specific document instead of the Internet or using external APIs. The chatbot was made with n8n, Ollama, PostgreSQL and pgvector.

The chatbot works by:

- Reading a PDF Document locally
- Splitting it into smaller chunks
- Converting chunks into embeddings and storing in PostgreSQL
- Retrieving the most relevant chunks related to user's question
- Passing those chunks to Ollama 3.2:3b to generate a grounded answer.

Stages of RAG

Load & Prepare

The PDF is loaded into n8n, its text is extracted and divided into smaller overlapping chunks.

Embed & Store

EmbeddingGemma converts each chunk into a numerical vector, which is stored with the text in PostgreSQL using pgvector.

Retrieve Relevant Context

The user's question is converted into an embedding and compared with the stored vectors to find the most relevant chunks.

Generate the Answer

Llama 3.2 receives the retrieved context and produces a clear, document-grounded response.

Stage 1: Load & Prepare

The source PDF is first loaded into n8n from the locally mounted documents folder.

The workflow then:

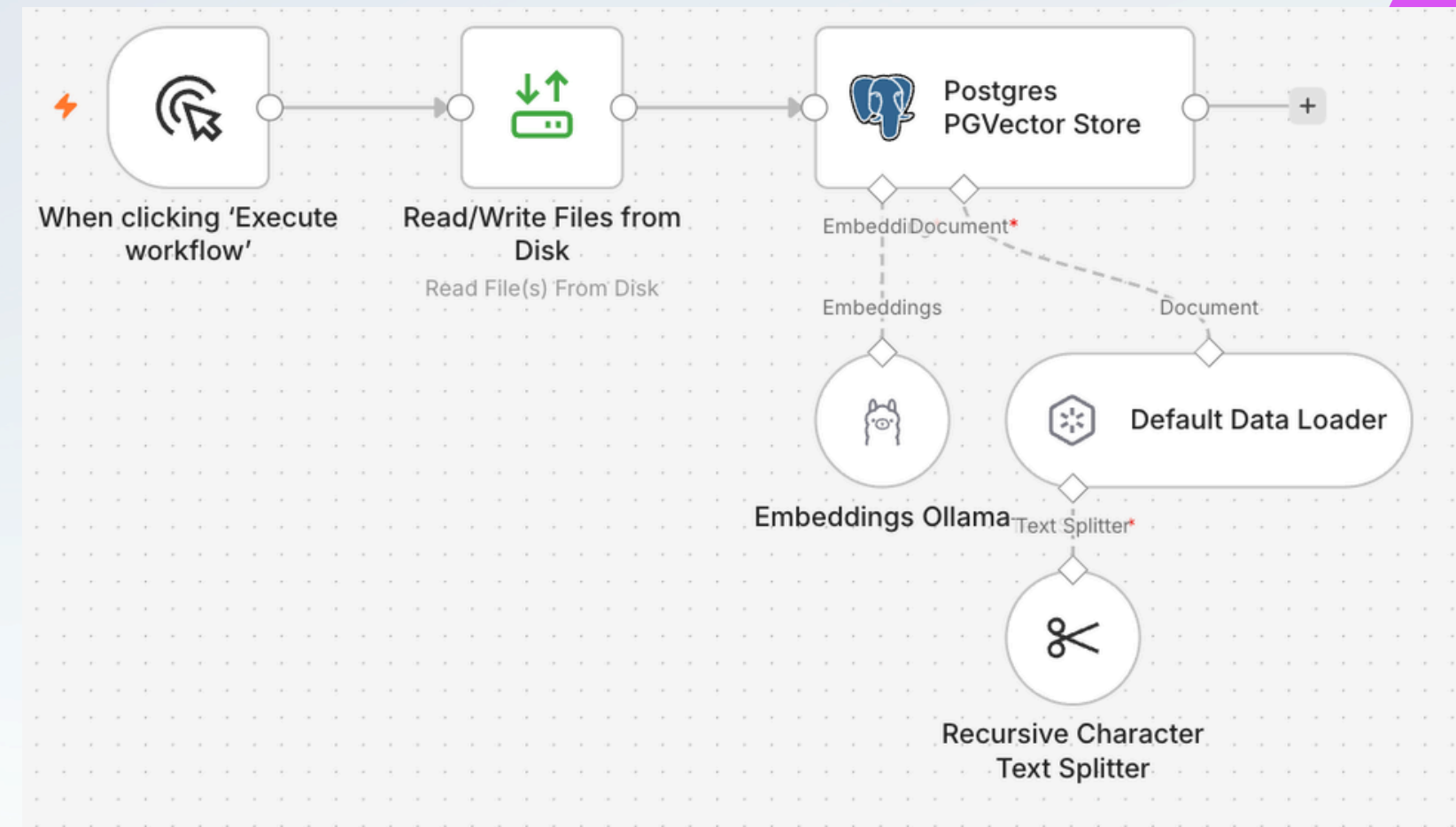
- Reads the binary PDF file
- Extracts readable text
- Divides the text into smaller overlapping chunks
- Prepares every chunk for embedding generation

Chunk settings used:

- Chunk size: 800 characters
- Chunk overlap: 120 characters

Why chunking is needed:

Smaller chunks allow the system to retrieve only the sections most relevant to the user's question.



Stage 2: Embed & Store

Each document chunk is converted into an embedding using the locally hosted EmbeddingGemma model.

An embedding is a numerical representation of the chunk's semantic meaning.

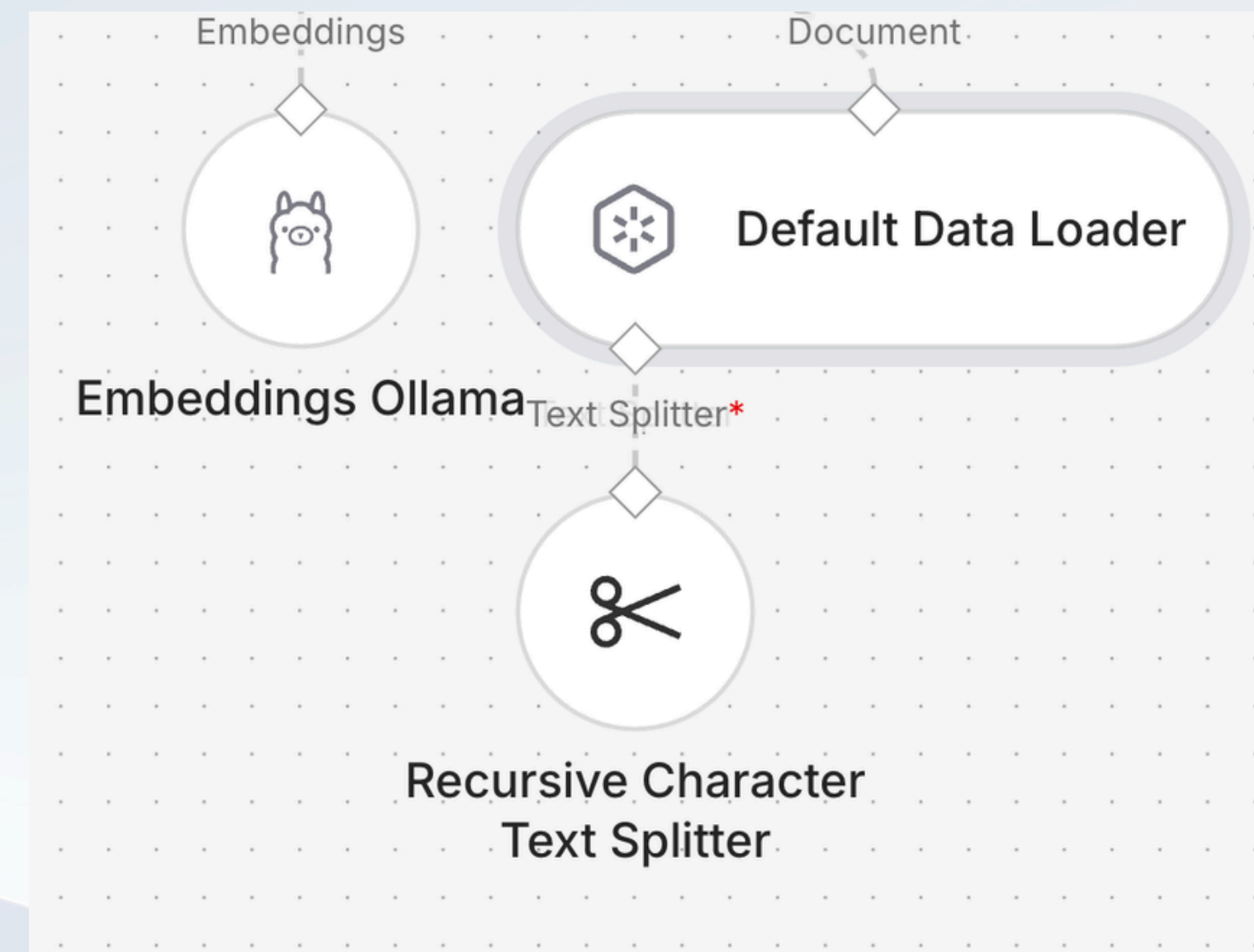
The system stores three main components in PostgreSQL:

- Original chunk text
- Numerical embedding
- Supporting document information

Parameters for Recursive Character Text Splitter:

Chunk Size: 800 and Chunk Overlap: 120

PostgreSQL uses the pgvector extension to store and compare these vectors.



Stage 3: Retrieve Relevant Context

When a user enters a question, the same embedding model converts the question into a vector.

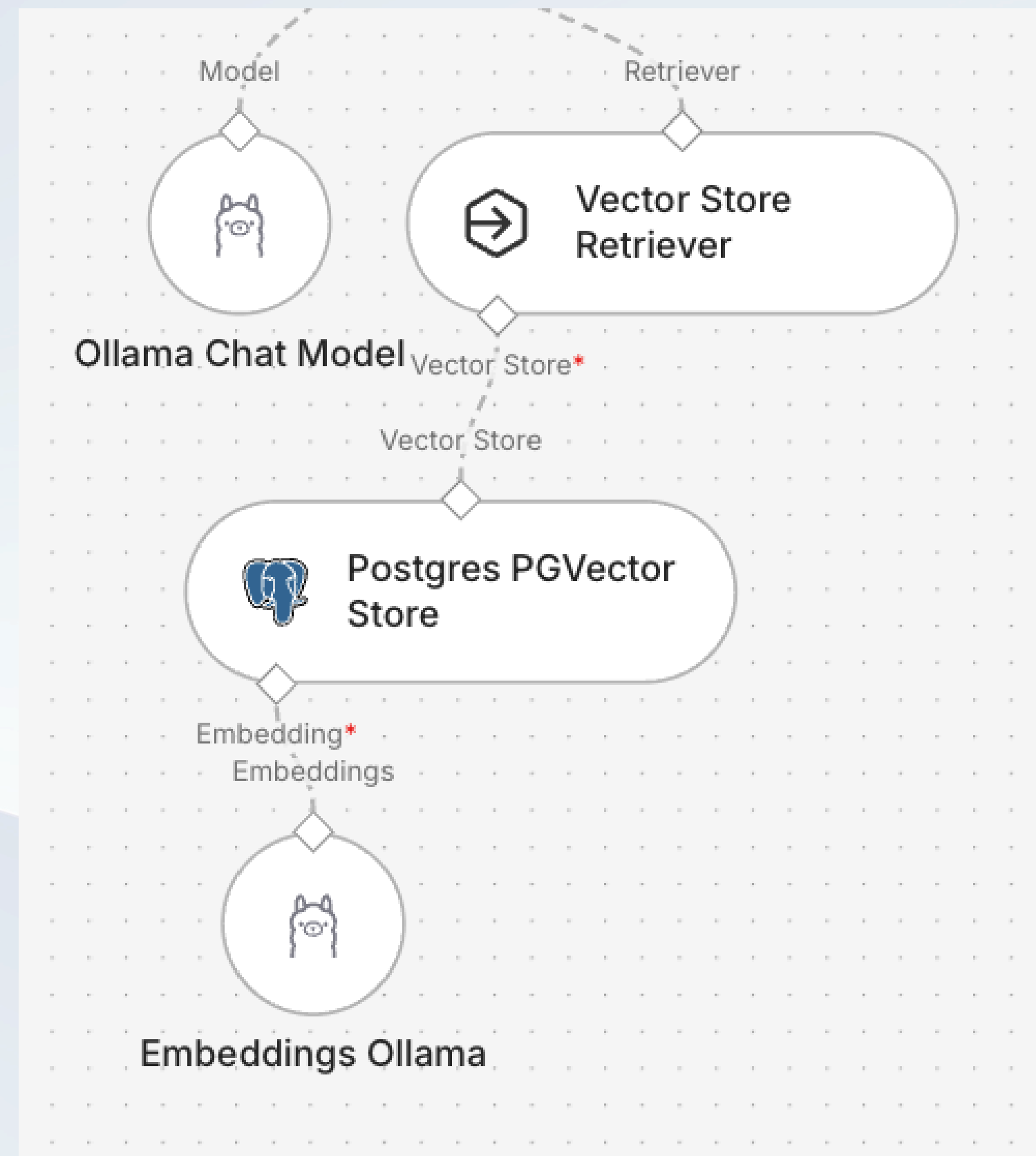
PostgreSQL compares the question vector against the 96 stored document vectors and retrieves the closest matching chunks.

The retrieval flow is: User question → Question embedding → Vector similarity search → Most relevant document chunks

The Vector Store Retriever controls how many chunks are returned to the language model.

Current retrieval configuration:

- PostgreSQL table: n8n_vectors
- Embedding model: embeddinggemma
- Top K: approximately 4 relevant chunks
- Metadata filters removed during initial testing



Stage 4: Generate the answer

The retrieved document chunks are inserted into the Question and Answer Chain as context.

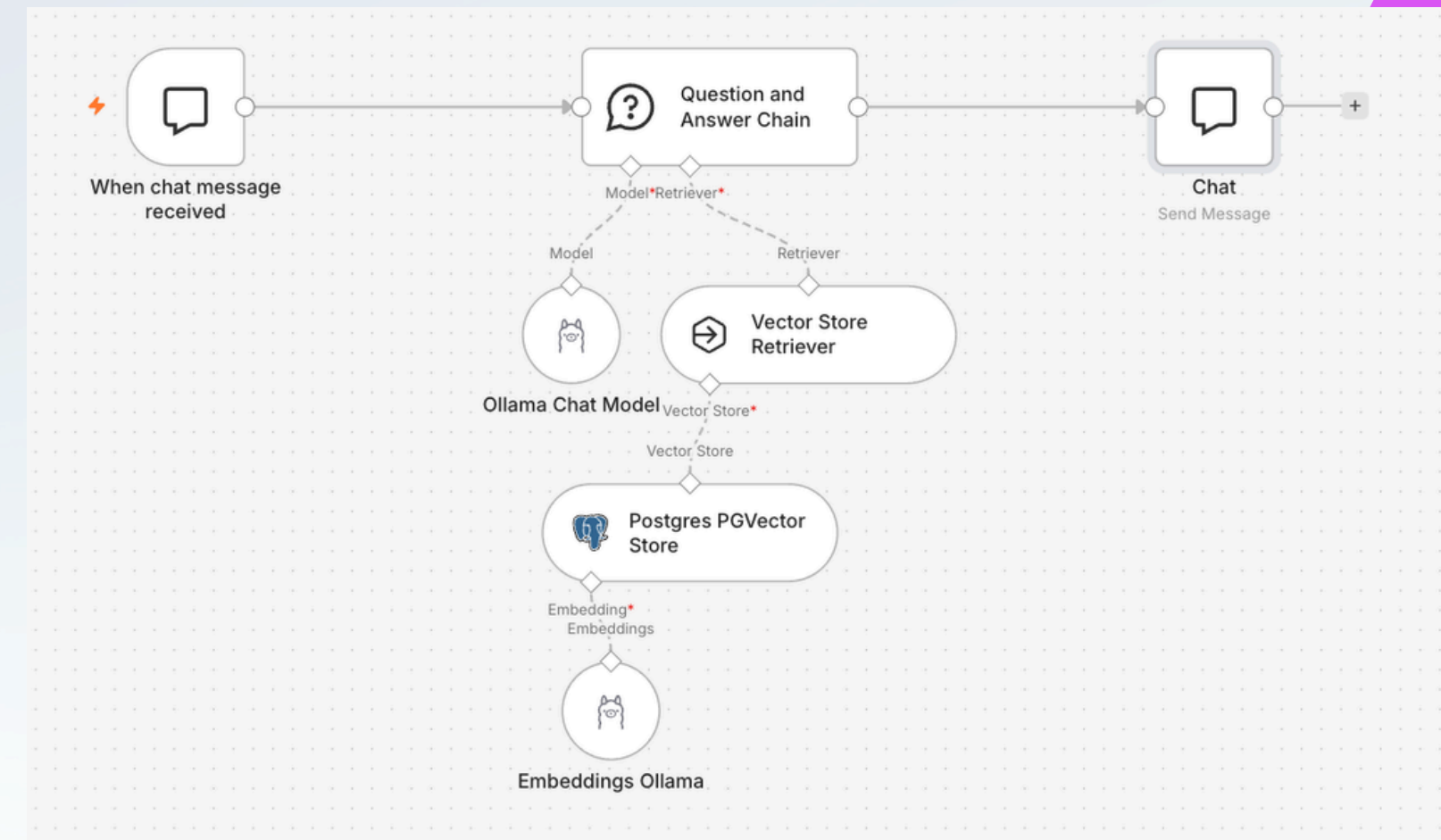
Llama 3.2 receives:

- The user's question
- The retrieved document context
- The system instructions

The model then generates a clear response based on the retrieved information.

Question + Retrieved Context → Llama 3.2 → Final Chatbot Answer

The prompt was configured to instruct the model to answer from the indexed document and avoid unsupported information.



Testings and Key Learnings

Tests performed:

- Asked questions directly covered by the indexed RAG paper
- Asked unrelated questions to test LLM fallback behaviour
- Compared general LLM answers with document-grounded answers
- Tested different retrieval limits
- Verified document chunks directly in PostgreSQL
- Tested metadata filtering and removed it during the initial version
- Corrected visible \n response-formatting issues

RAG does not replace an LLM. It adds a retrieval layer before the LLM. Retriever finds the information and the LLM explains the information. When asking the chatbot questions irrelevant to the document it states it cannot answer the question as shown in the image.

What is RAG?

RAG stands for Retrieval-Augmented Generation (RAG), a model that combines retrieval-based approaches with generation flexibility, allowing it to generate answers even when the answer is not verbatim in the retrieved documents.

What is the capital of Islamabad

I could not find this information in the indexed document.

Final Product

The project produced a functioning local RAG chatbot capable of retrieving information from an indexed document and using it as context for an AI-generated response. The chatbot's capabilities:

- Fully local operation - No external AI APIs - Semantic document search - PostgreSQL based vector storage - Natural-language question answering - Modular n8n workflows
- Foundation for additional documents and company knowledge

The completed system transforms a static PDF into an interactive knowledge source that users can query through a chatbot.